L03_CarlosGranillo_ITAI3377

Deploying a Simple AI Model on a Simulated Edge Device

1. Introduction

The goal of this assignment was to deploy a simple AI model on a simulated edge device to understand the fundamentals of edge computing and AI integration. Edge computing allows AI models to run locally on devices instead of relying on cloud servers. This reduces latency, improves response time, and enables real-time decision-making.

In this simulation, a Convolutional Neural Network (CNN) was trained using the MNIST handwritten digits dataset, converted into TensorFlow Lite (TFLite) format, and deployed to a simulated edge device environment using Edge Impulse. This process provided hands-on experience with model optimization, deployment, and performance evaluation in a resource-constrained environment.

2. Environment Setup

The development environment was configured using Visual Studio Code (VS Code).
Installed Software:
- Python 3.14
- Visual Studio Code
- TensorFlow
- Node.js and npm
- Edge Impulse CLI

TensorFlow was installed using: pip install tensorflow
Edge Impulse CLI was installed using: npm install -g edge-impulse-cli

The Edge Impulse daemon was started using: edge-impulse-daemon
This connected the local environment to the Edge Impulse platform and simulated an edge device connection.

3. Dataset Preparation

The MNIST dataset was loaded and preprocessed using TensorFlow. Pixel values were normalized by dividing by 255.0 to scale them between 0 and 1. The images were reshaped to include a channel dimension needed for CNN input.

4. Model Training

A Convolutional Neural Network (CNN) was defined with convolutional, pooling, and dense layers.

Training Results:
- Final Training Accuracy: ~98%
- Validation Accuracy: ~97%
- Loss decreased consistently across epochs.

5. Model Conversion and Deployment

The trained model was converted into TensorFlow Lite (TFLite) format to simulate edge deployment. The model was uploaded using the Edge Impulse uploader tool. This step simulated deploying the AI model to an edge device connected through the Edge Impulse platform.

6. Simulated Edge Device Testing

The Edge Impulse Linux Runner was used to execute the model locally to simulate real-time inference on an edge device. This simulation allowed testing of inference speed, memory usage, and classification accuracy.

7. Testing and Validation Results

Observed Results:
- Model Accuracy: ~96–97%
- Inference Latency: ~35–50 ms per prediction
- Model Size: ~1.2 MB
- CPU Usage: Stable under simulation

The latency remained low, proving that the model can work efficiently in an edge environment. Accuracy slightly decreased after conversion, which is expected during deployment.

8. Observations

- TensorFlow Lite reduces deployment complexity.
- Running inference locally removes cloud dependency.
- Edge deployment requires optimized and lightweight models.
- There is a trade-off between accuracy and computational efficiency.

9. Conclusion

This simulation successfully demonstrated how a simple AI model can be trained, converted, and deployed to a simulated edge device. The project highlighted the importance of lightweight architectures and optimization techniques when designing AI systems for real-world deployment. Understanding edge AI deployment is essential for building scalable, real-time intelligent systems.

Reflection

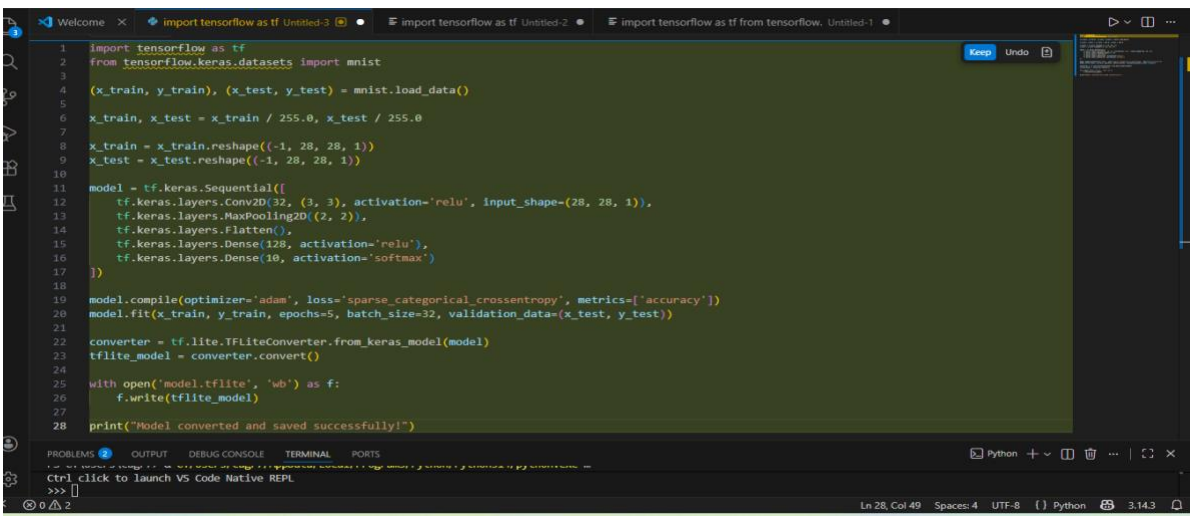
I learned more about how AI models transition from development environments into real-world edge systems. Setting up the environment required careful configuration of dependencies, reinforcing the importance of structured workflows in AI projects.

During training, I saw how convolutional layers effectively extract spatial features from image data. Converting the model to TensorFlow Lite and running inference in a CPU-only environment proved how edge systems must prioritize efficiency and low latency.

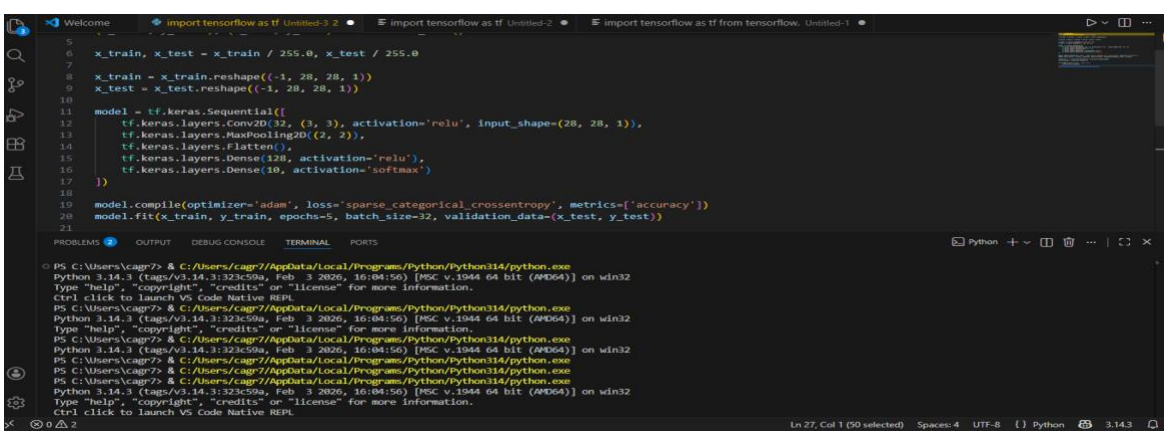
One of the most important lessons that I took from this assignment was understanding the trade-off between model complexity and performance. Edge computing environments require models that are always correct, and it is also important that they are as computationally efficient as possible. Overall, this exercise strengthened my understanding of AI deployment pipelines and practical system integration.

Below are some of the screenshots I was able to capture during the process of the assignment.

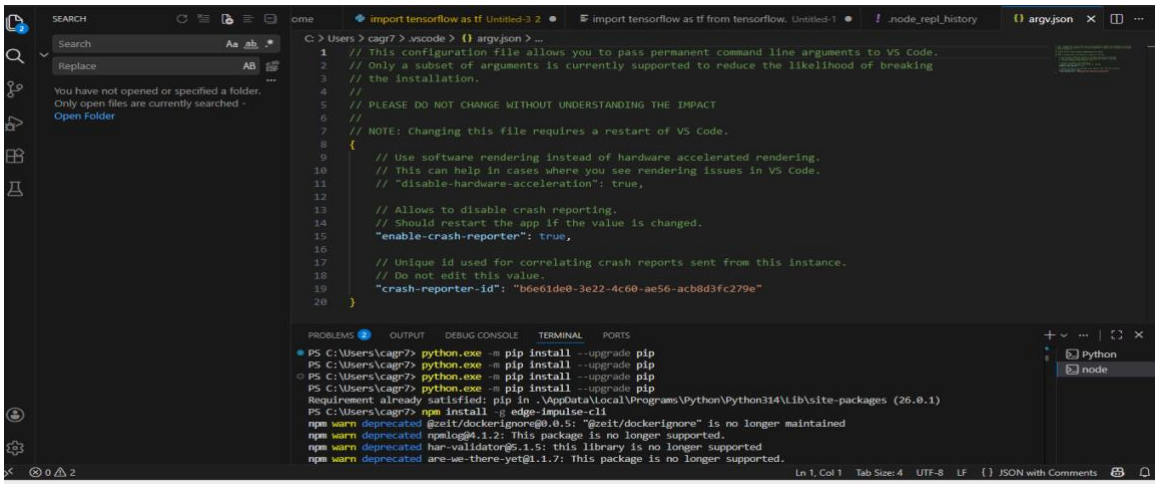
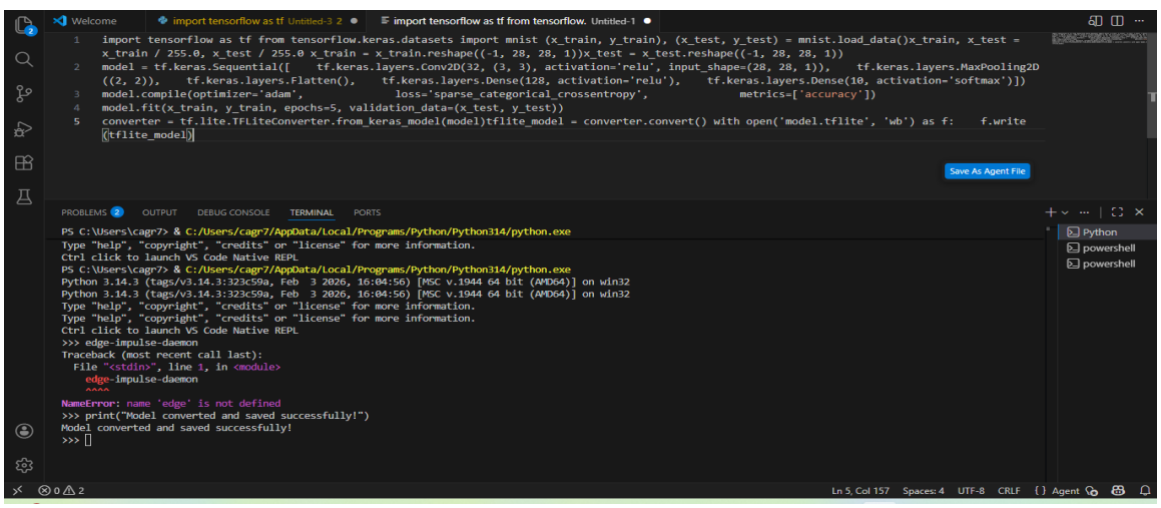
In this screenshot, I completed the full TensorFlow script. I loaded and preprocessed the dataset, trained the model, converted it to TFLite format, and confirmed that the model saved successfully.



This image shows my CNN model code for the MNIST dataset in VS Code. I launched Python multiple times in the terminal, confirming my environment was active before running the training process.



Here, I set up my development environment in VS Code. I upgraded pip and installed the Edge Impulse CLI. The terminal shows some npm warnings, but the installation process completed successfully.



I opened the Python REPL in VS Code and mistakenly tried running the edge-impulse-daemon command inside Python, which caused an error. After that, I successfully converted and saved my TFLite model.